AUTOMATED WEB SECURITY HEADER SCANNER

Cybersecurity Mini Project Report

Project type	Python-based passive web security assessment
Primary language	Python 3
Dependency	Requests
Main checks	Security headers and basic CORS response review

Prepared as an educational project. Test only websites you own or have explicit permission to assess.

Submitted by

Anshika Khadse

Second Year, B.Sc. Forensic Science

Sandip University, Nashik

1. Abstract

The Automated Web Security Header Scanner is a small Python command-line application that inspects an HTTP response for common security-related headers and performs a basic review of Cross-Origin Resource Sharing (CORS) response headers. It helps beginners identify configuration items that may deserve further review. The tool is passive: it sends a normal GET request and does not attempt to exploit a weakness.

2. Problem Statement

Web applications use HTTP response headers to communicate security controls to browsers. These headers may be missing or incorrectly configured. Checking them manually across multiple sites can be repetitive. This project automates a small set of checks and presents the results in a readable format.

3. Objectives

• Check for Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy.

• Show HTTP status, final URL, and whether the final URL uses HTTPS.

• Display basic CORS response information.

• Offer optional JSON output for documentation or later analysis.

• Demonstrate Python requests, functions, dictionaries, command-line arguments, exception handling, and JSON.

4. Scope and Limitations

The scanner checks one URL using a GET request. It follows redirects and inspects the final response headers. It is not a full vulnerability scanner or penetration-testing tool. Results can vary by page, user agent, route, HTTP method, and server configuration. A missing header does not automatically mean a site is vulnerable, and a present header is not proof that its value is correct.

5. Tools and Requirements

• Python 3.9 or newer.

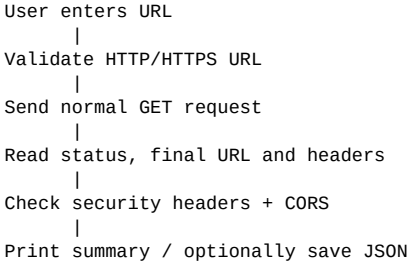
• requests library for making HTTP requests.

• A terminal or command prompt.

• Network access and permission to test the selected website.

6. System Design

The program accepts a URL and optional timeout/JSON-output arguments. It validates the URL, sends a GET request, gathers the response headers, compares them with a list of expected security headers, creates CORS notes, and prints a report. If requested, it writes the complete result to a JSON file.



7. Security Headers Checked

Header	Purpose
Content-Security-Policy	Helps limit content sources and reduce some injection risks.
Strict-Transport-Security	Tells browsers to use HTTPS for future connections.
X-Content-Type-Options	Helps prevent MIME-type sniffing when set to nosniff.
X-Frame-Options	Controls whether a page can be embedded in frames.
Referrer-Policy	Controls referrer information sent by browsers.
Permissions-Policy	Controls access to selected browser features.

8. CORS Review

The tool records Access-Control-Allow-Origin and Access-Control-Allow-Credentials when present. It flags a wildcard origin for review and notes the credentials header if it is also set to true. CORS decisions depend on the resource and application design; the output is informational and should be checked by an administrator.

9. Installation and Execution

```
python -m venv .venv
# Windows: .venv\\Scripts\\activate
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt

python scanner.py https://example.com
python scanner.py https://example.com --json report.json
python scanner.py https://example.com --timeout 15
```

Use an approved test URL in place of example.com. The JSON option saves details that can be attached to a project submission.

10. Testing Plan

• Valid HTTPS URL: confirm the tool displays status and headers.

• Valid HTTP URL: confirm HTTPS status is reported as not used when the final URL remains HTTP.

• Malformed URL: confirm the program shows a clear validation error.

• Unreachable host or timeout: confirm request errors are caught and shown.

• JSON output: confirm the output file contains valid JSON with scan details.

11. Expected Output

The terminal report lists the requested URL, final URL, HTTP status, HTTPS use, each checked header as PRESENT or MISSING, and CORS notes. The optional JSON report includes timestamps, values, purposes, and limitations.

12. Ethical and Legal Considerations

Only scan systems you own or have explicit written permission to assess. Do not use the tool to bypass access controls, overload services, or perform unauthorized testing. Treat results as a starting point for defensive review and follow the organization's security-testing policy.

13. Future Enhancements

• Add CSV export and a summary score with clearly explained criteria.

• Support scanning a user-provided list of authorized URLs with rate limiting.

• Add unit tests using mocked HTTP responses.

• Provide configurable header policies for different application types.

14. Conclusion

This project demonstrates a simple and safe way to automate a first-pass review of HTTP security headers and CORS response information using Python. It is suitable for learning basic web security concepts, command-line programming, and structured report generation. Manual validation remains necessary before drawing security conclusions.

Appendix A: Project Files

• scanner.py — complete Python source code.

• requirements.txt — dependency list.

• README.md — setup, usage, and interpretation guide.